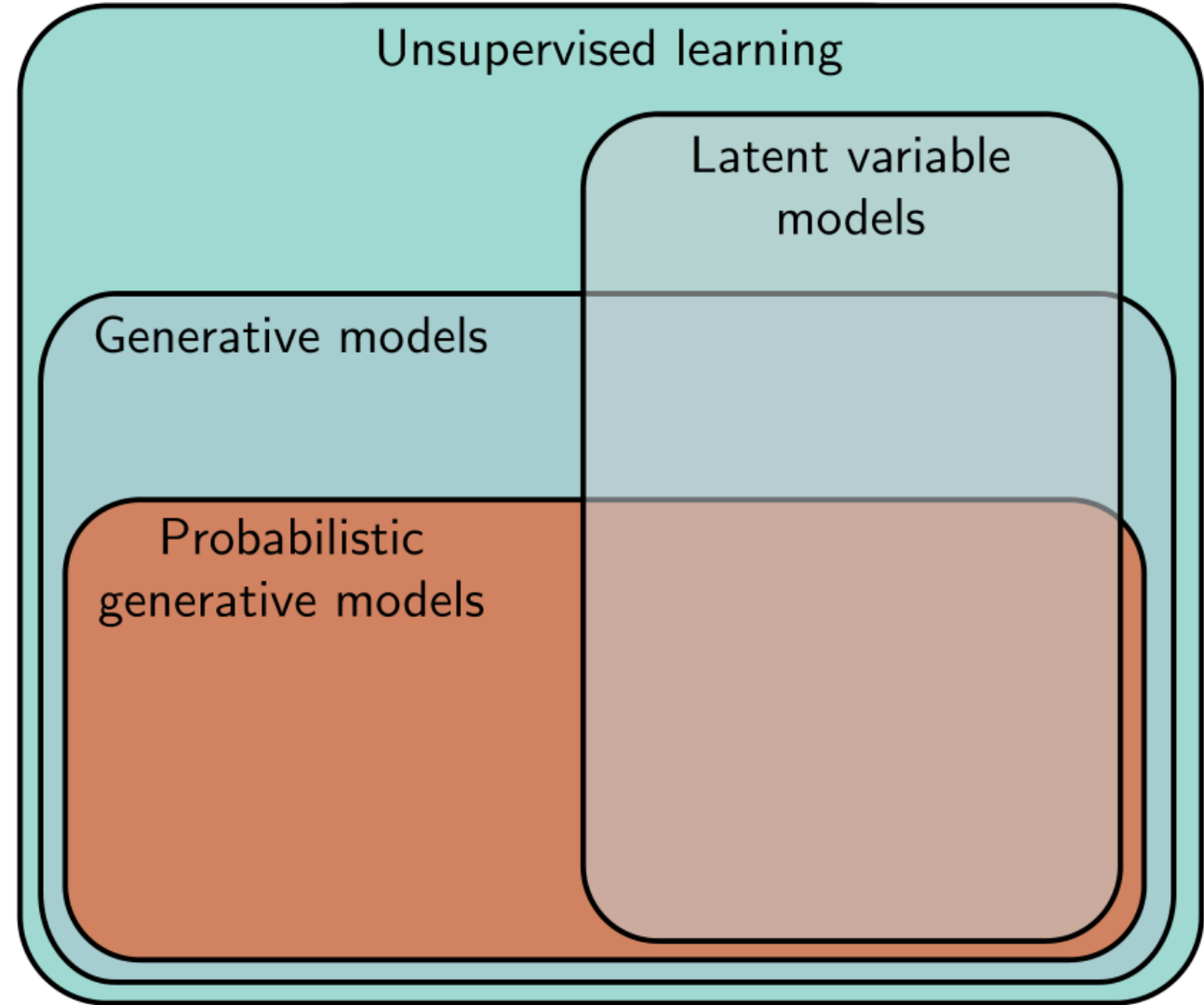


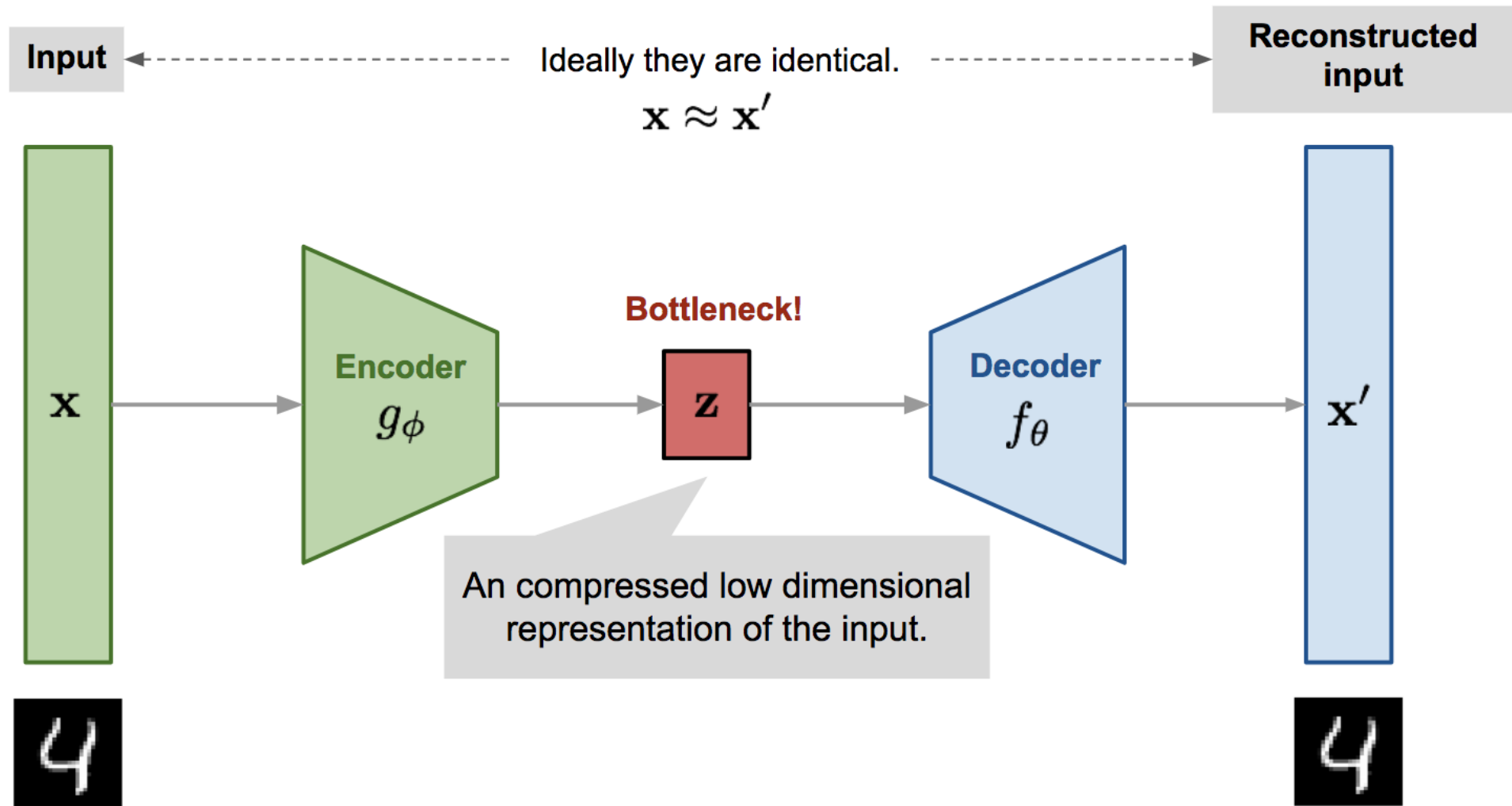
Generative Models

Unsupervised Learning

- Generative models – can synthesize (generate) new examples with similar statistics to the training data
- Probabilistic generative models – learn a probability distribution over the training data
- Latent variable models – define a mapping between an underlying explanatory (latent) variable and the data

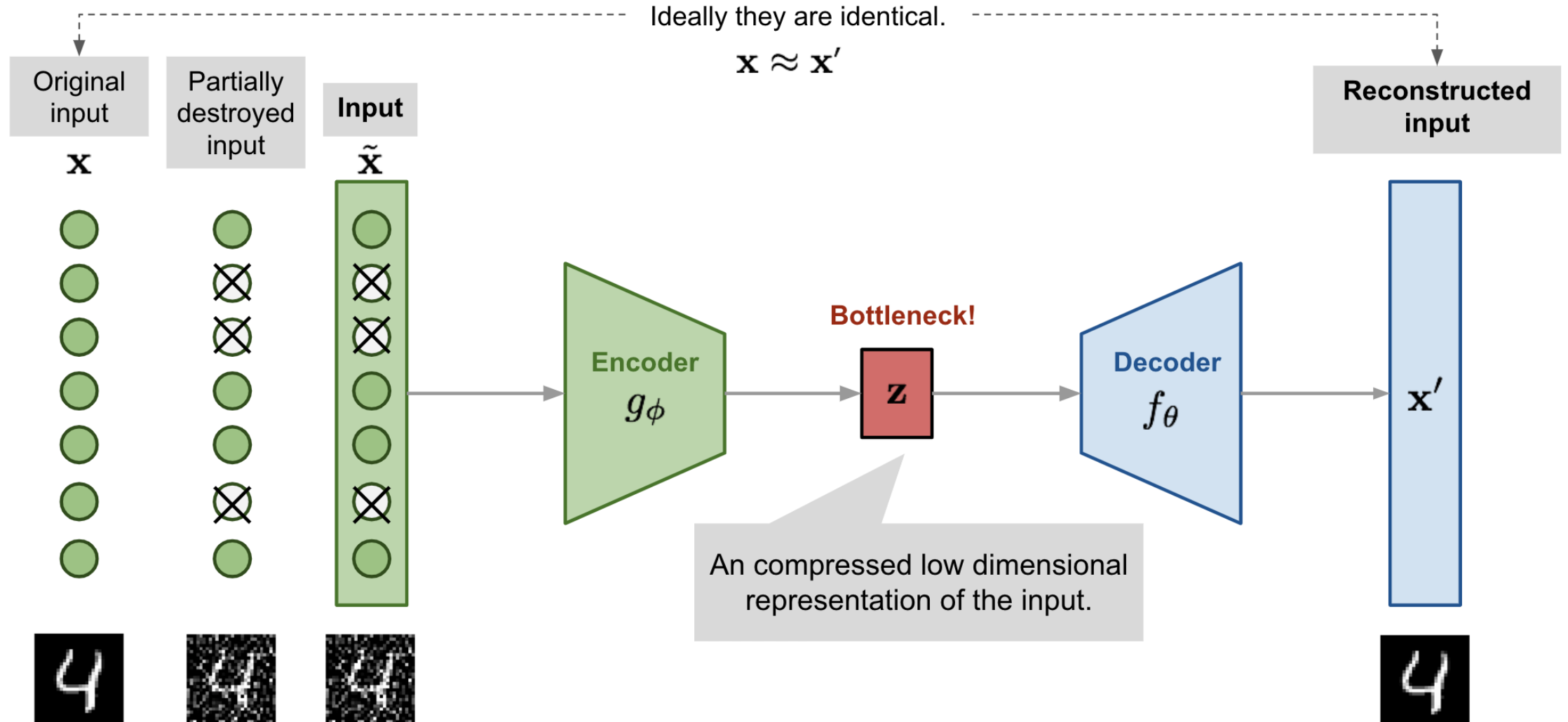


Autoencoder

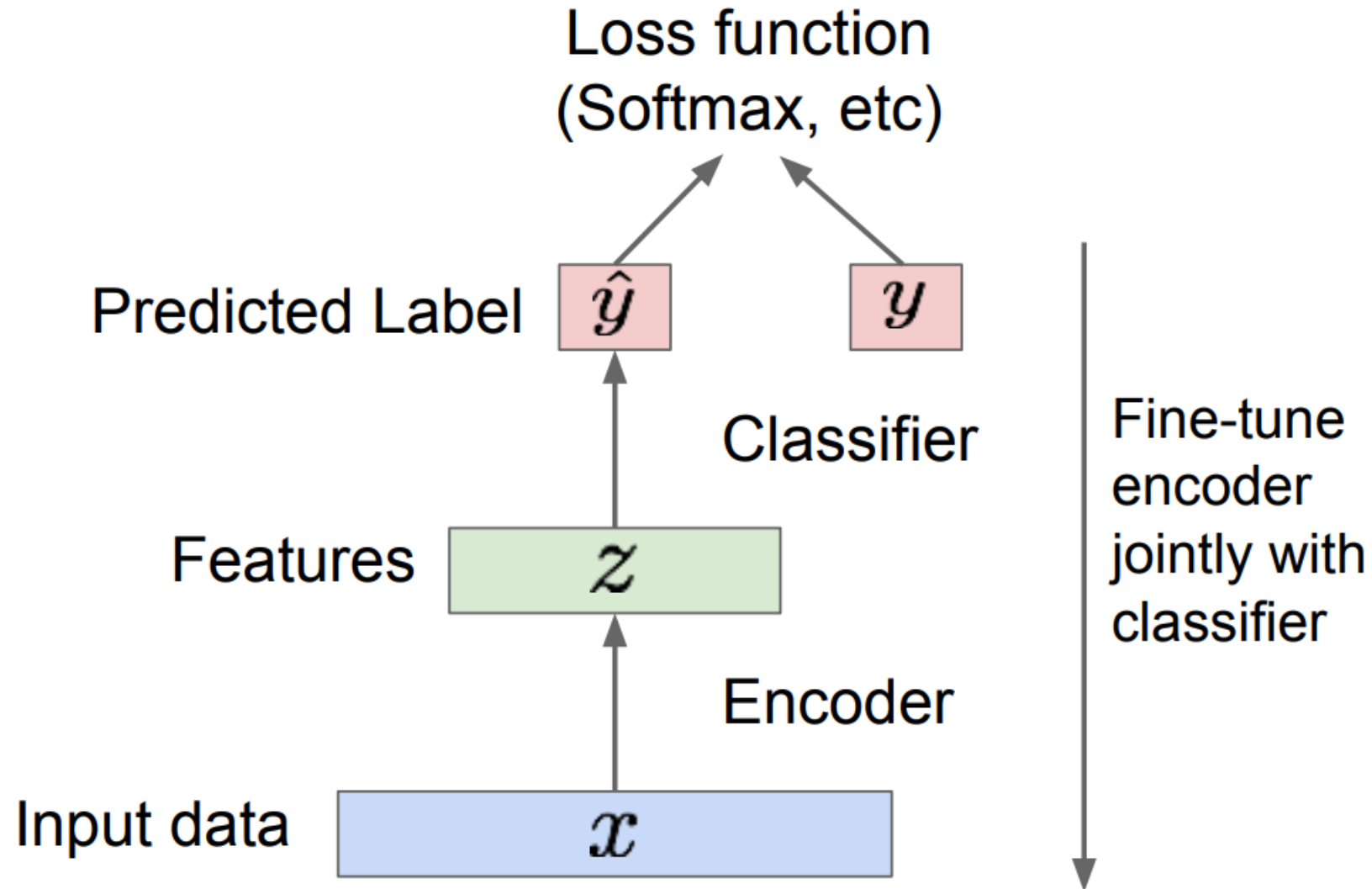


$$L_{\text{AE}}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}^{(i)} - f_\theta(g_\phi(\mathbf{x}^{(i)})))^2$$

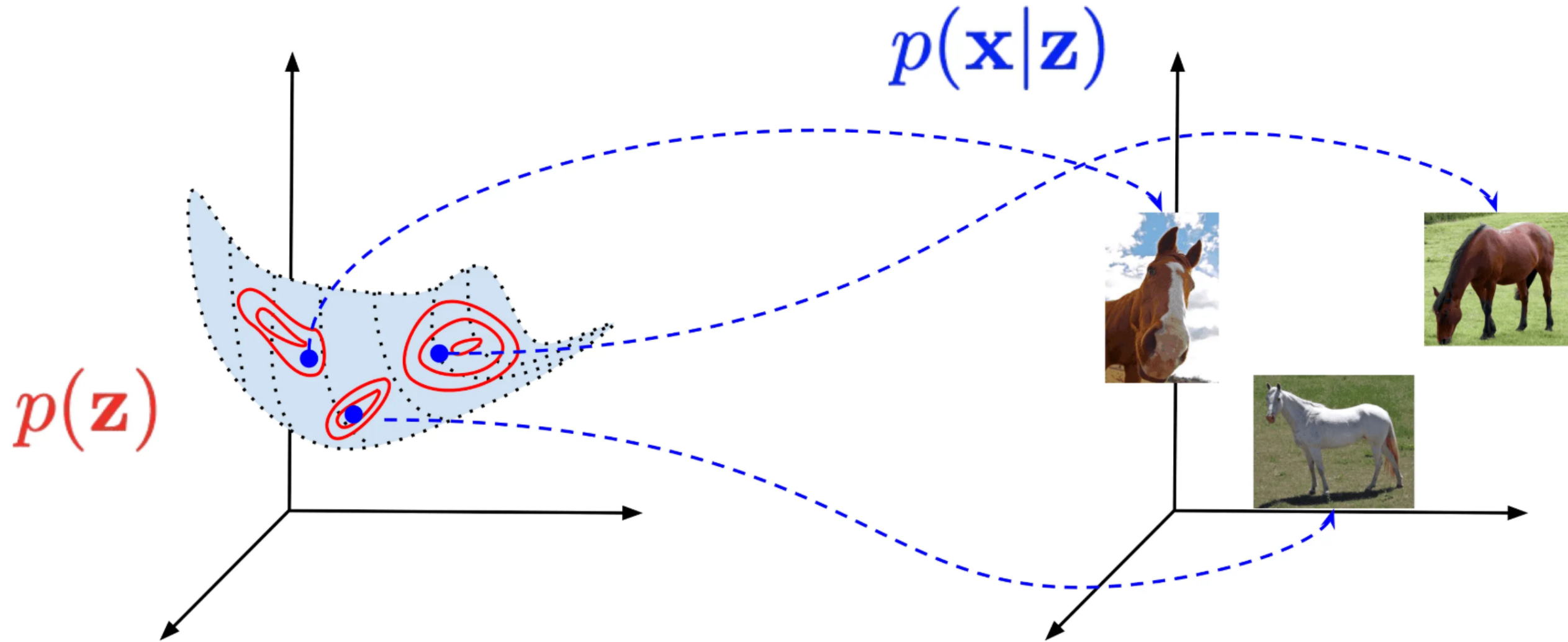
Denoising Autoencoder



Autoencoder: Usage



VAE: Variational Autoencoder



VAE: Variational Autoencoder

Instead of mapping the input into a **fixed** vector, we want to map it into a **distribution** p_{θ}

The relationship between the data input $\mathbf{x}$ and the latent encoding vector $\mathbf{z}$ can be fully defined by:

- Prior $p_{\theta}(\mathbf{z})$
- Likelihood $p_{\theta}(\mathbf{x}|\mathbf{z})$
- Posterior $p_{\theta}(\mathbf{z}|\mathbf{x})$

If we know the real θ^* , we can sample that look like a real data point $\mathbf{x}^{(i)}$:

- Sample a $\mathbf{z}^{(i)}$ from a prior distribution $p_{\theta^*}(\mathbf{z})$
- Generate $\mathbf{x}^{(i)}$ from $p_{\theta^*}(\mathbf{x}|\mathbf{z} = \mathbf{z}^{(i)})$

VAE: Variational Autoencoder

Target: maximizes the probability of generating real data samples

$$\theta^* = \arg \max_{\theta} \prod_{i=1}^n p_{\theta}(\mathbf{x}^{(i)}) = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}^{(i)})$$

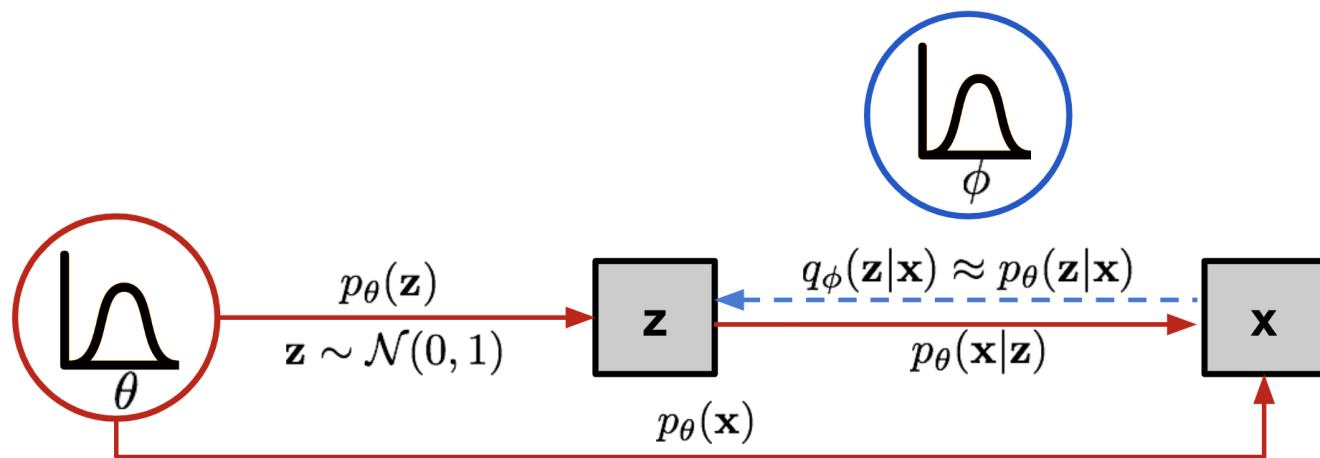
But $p_{\theta}(\mathbf{x}^{(i)})$ **intractable** to compute!

$$p_{\theta}(\mathbf{x}^{(i)}) = \int p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})p_{\theta}(\mathbf{z})d\mathbf{z}$$

Posterior density is also **intractable**:

$$p_{\theta}(\mathbf{z}|\mathbf{x}) = \frac{p_{\theta}(\mathbf{x}|\mathbf{z})p_{\theta}(\mathbf{z})}{p_{\theta}(\mathbf{x})}$$

VAE: Loss Function



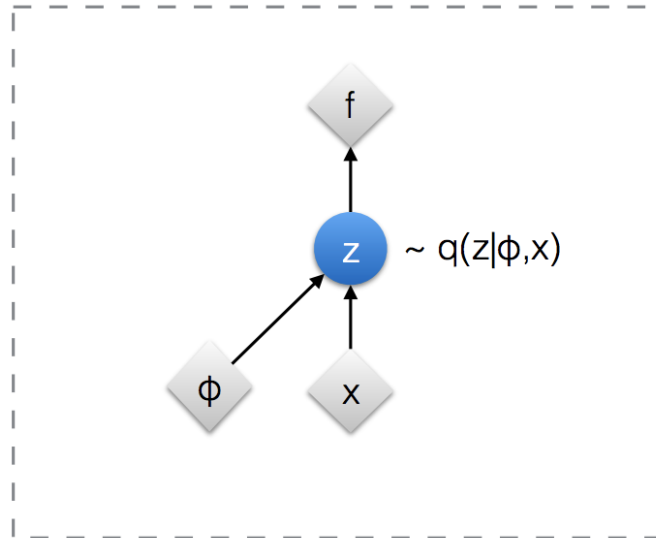
$$D_{KL} \left(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p_\theta(\mathbf{z}|\mathbf{x}) \right) = \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z}|\mathbf{x})} d\mathbf{z} =$$

$$= \log p_\theta(\mathbf{x}) + D_{KL} \left(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p_\theta(\mathbf{z}) \right) - \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z}) \Rightarrow$$

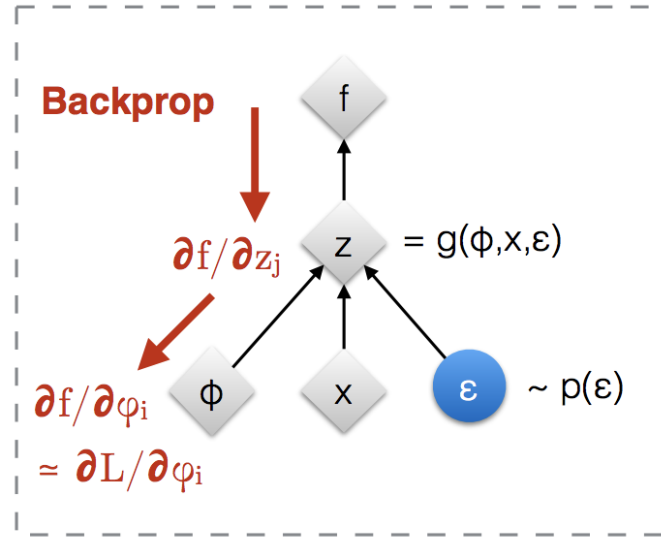
$$\log p_\theta(\mathbf{x}) - D_{KL} \left(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p_\theta(\mathbf{z}|\mathbf{x}) \right) = \underbrace{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z})}_{\text{Reconstruction term}} - \underbrace{D_{KL} \left(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p_\theta(\mathbf{z}) \right)}_{\text{Regularization term}}$$

VAE: Reparameterization Trick

Original form



Reparameterised form



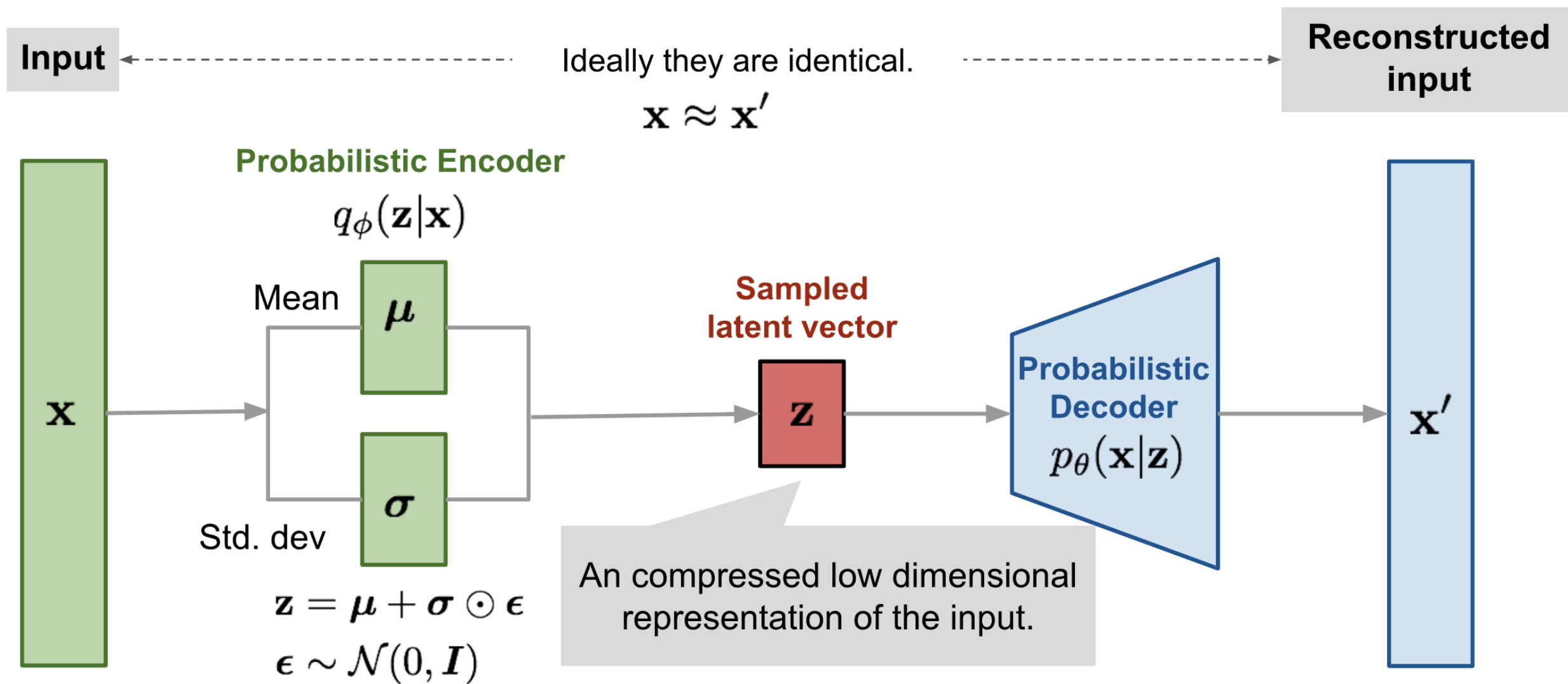
 : Deterministic node
 : Random node

[Kingma, 2013]
 [Bengio, 2013]
 [Kingma and Welling 2014]
 [Rezende et al 2014]

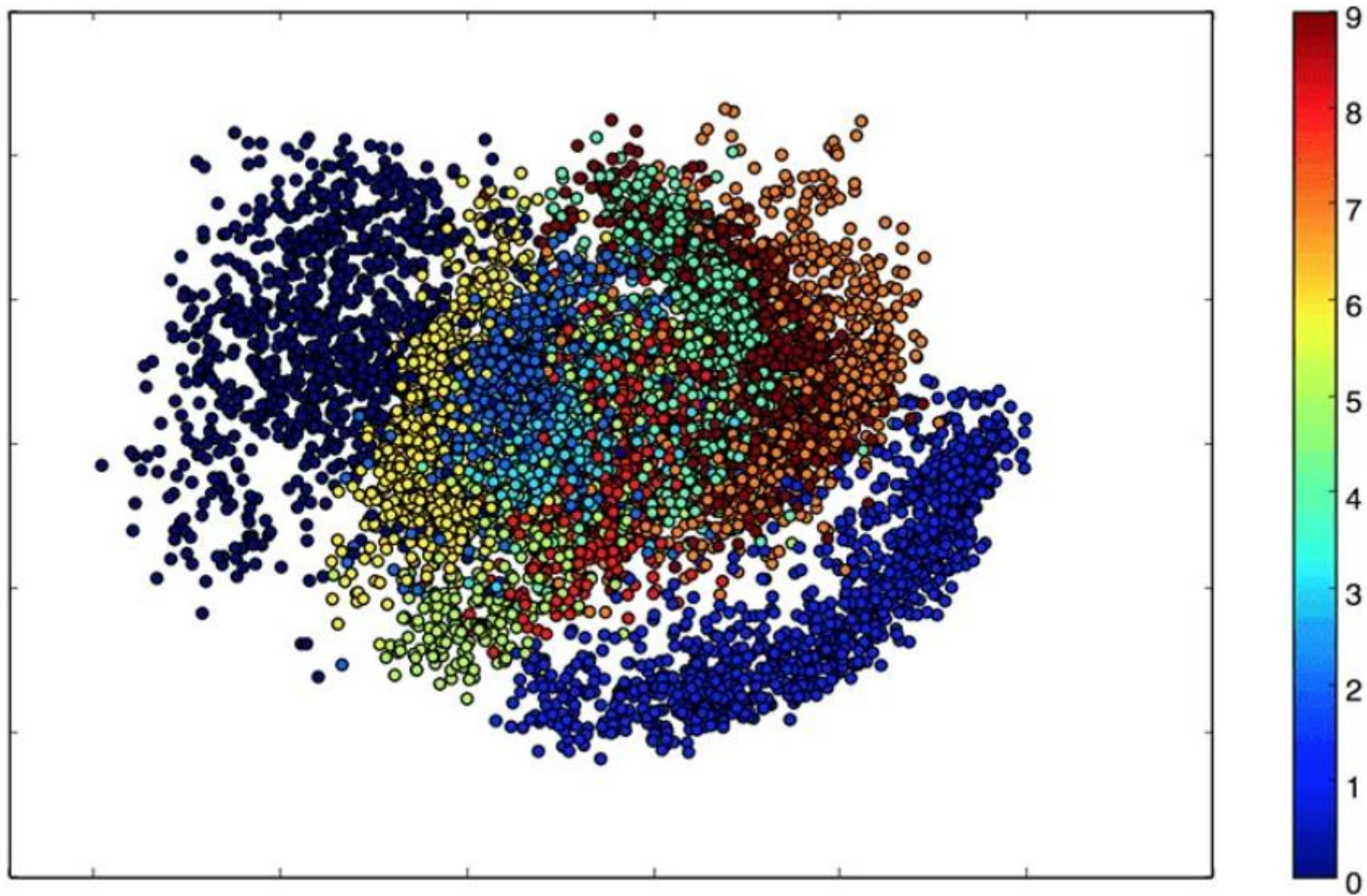
$$\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2(i)} \mathbf{I})$$

Reparameterization trick: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$

VAE



VAE: MNIST



VAE: MNIST

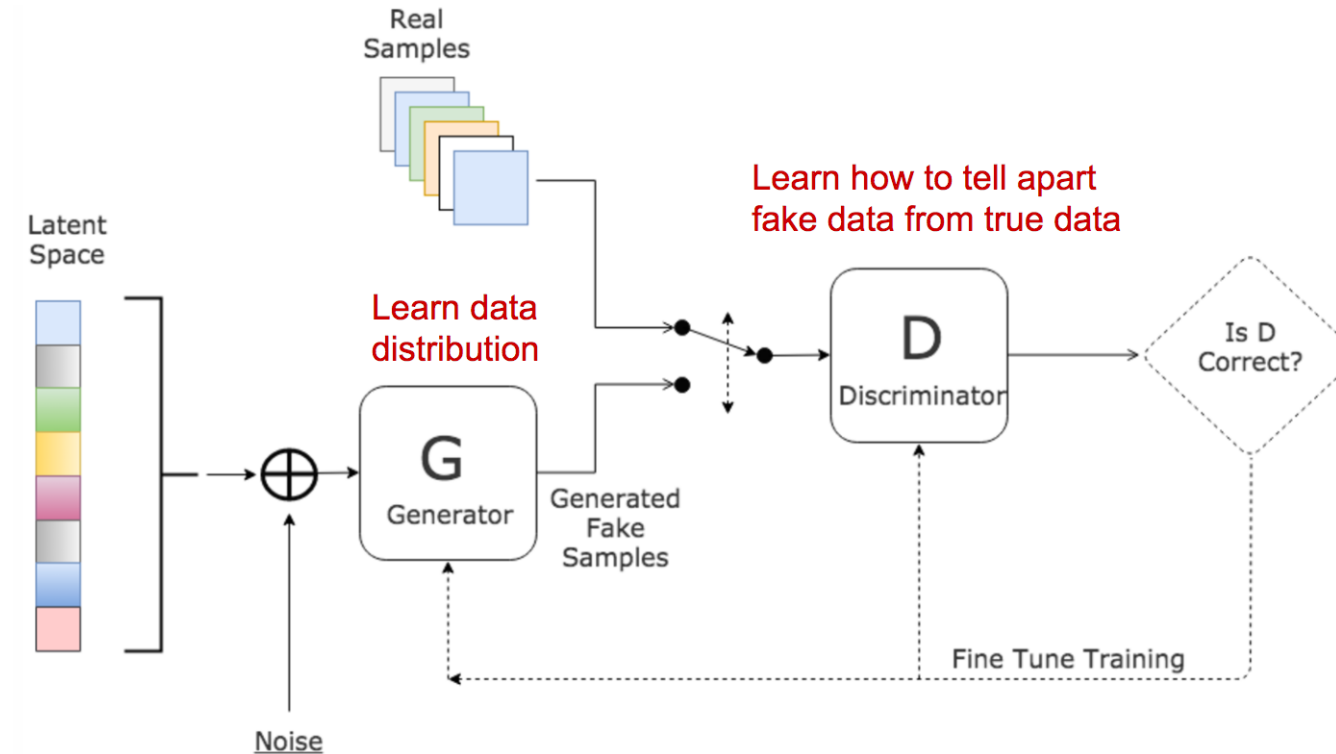


(a) 2-D latent space (b) 5-D latent space (c) 10-D latent space (d) 20-D latent space

VAE: Examples



GAN: Generative Adversarial Network



- A discriminator ***D*** estimates the probability of a given sample coming from the real dataset. It works as a critic and is optimized to tell the fake samples from the real ones.
- A generator ***G*** outputs synthetic samples given a noise variable input ***z***.

It's zero-sum game between two models motivates both to improve their functionalities

GAN: Loss

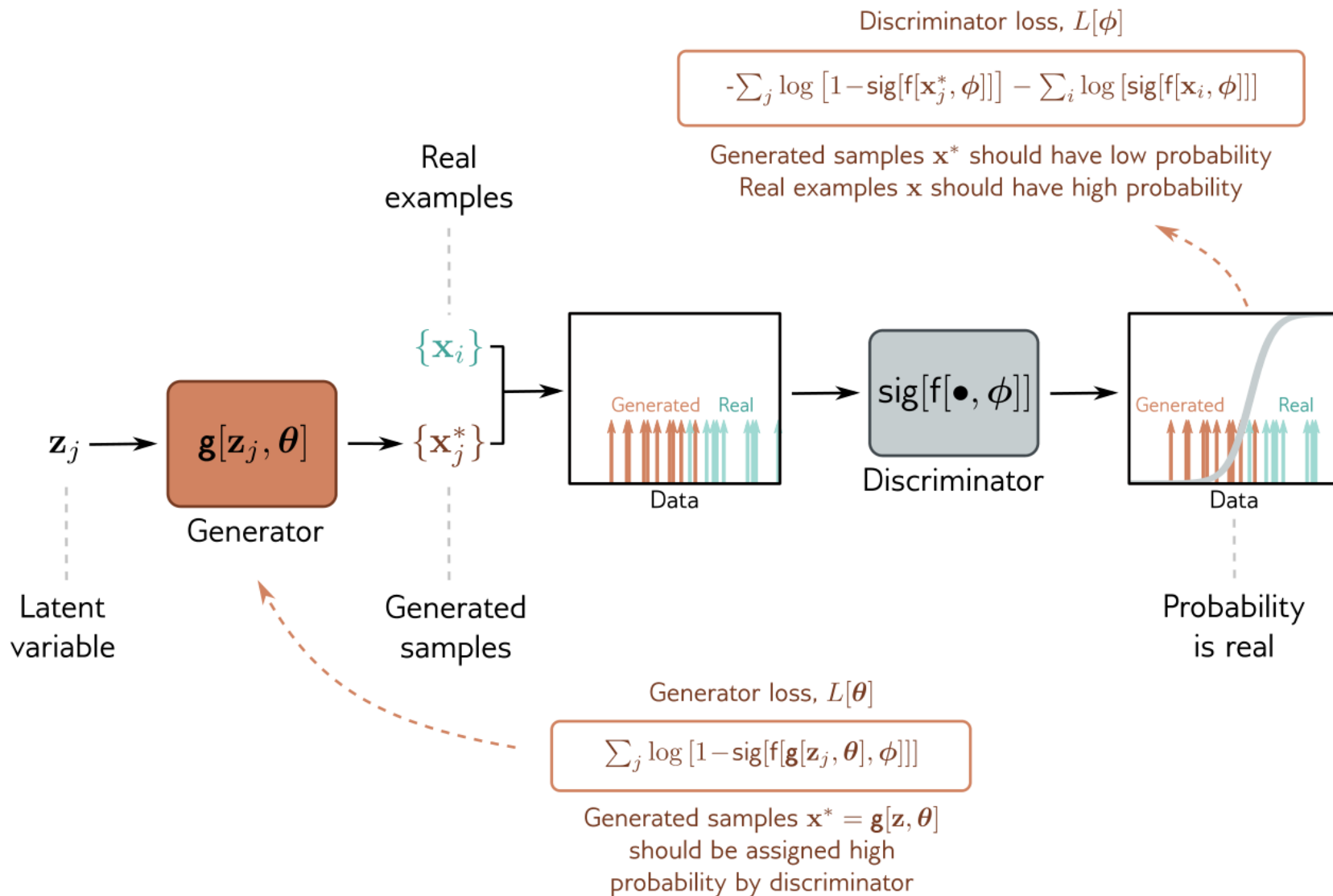
$$\min_G \max_D L(D, G) = \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] = \\ \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{x \sim p_g(x)} [\log(1 - D(x))]$$

p_z — data distribution over noise input z (usually, just uniform)

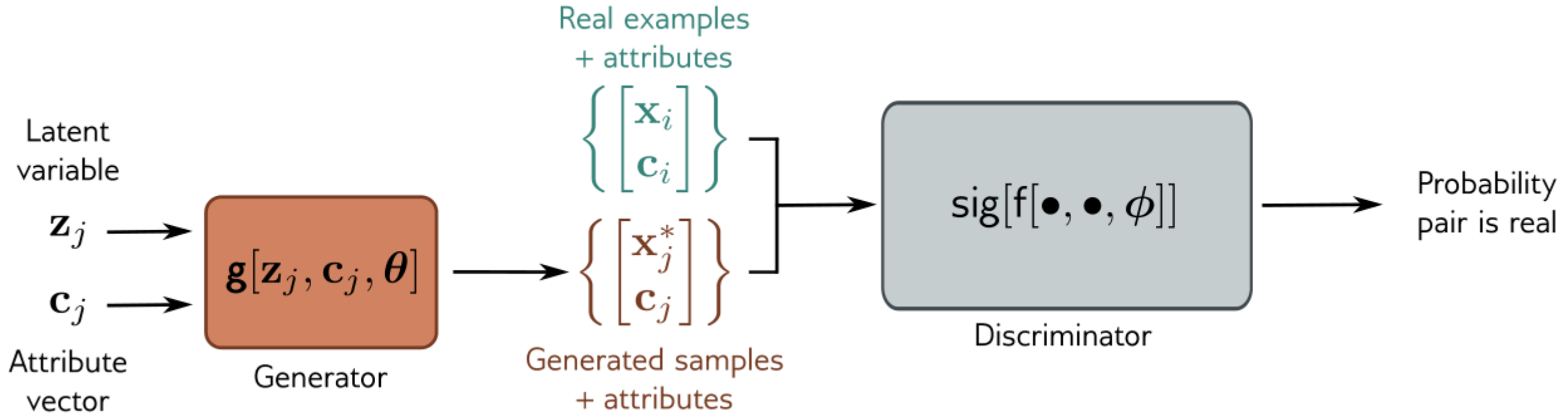
p_g — the generator's distribution over data x

p_r — data distribution over real sample x

GAN: Training



Conditional GAN

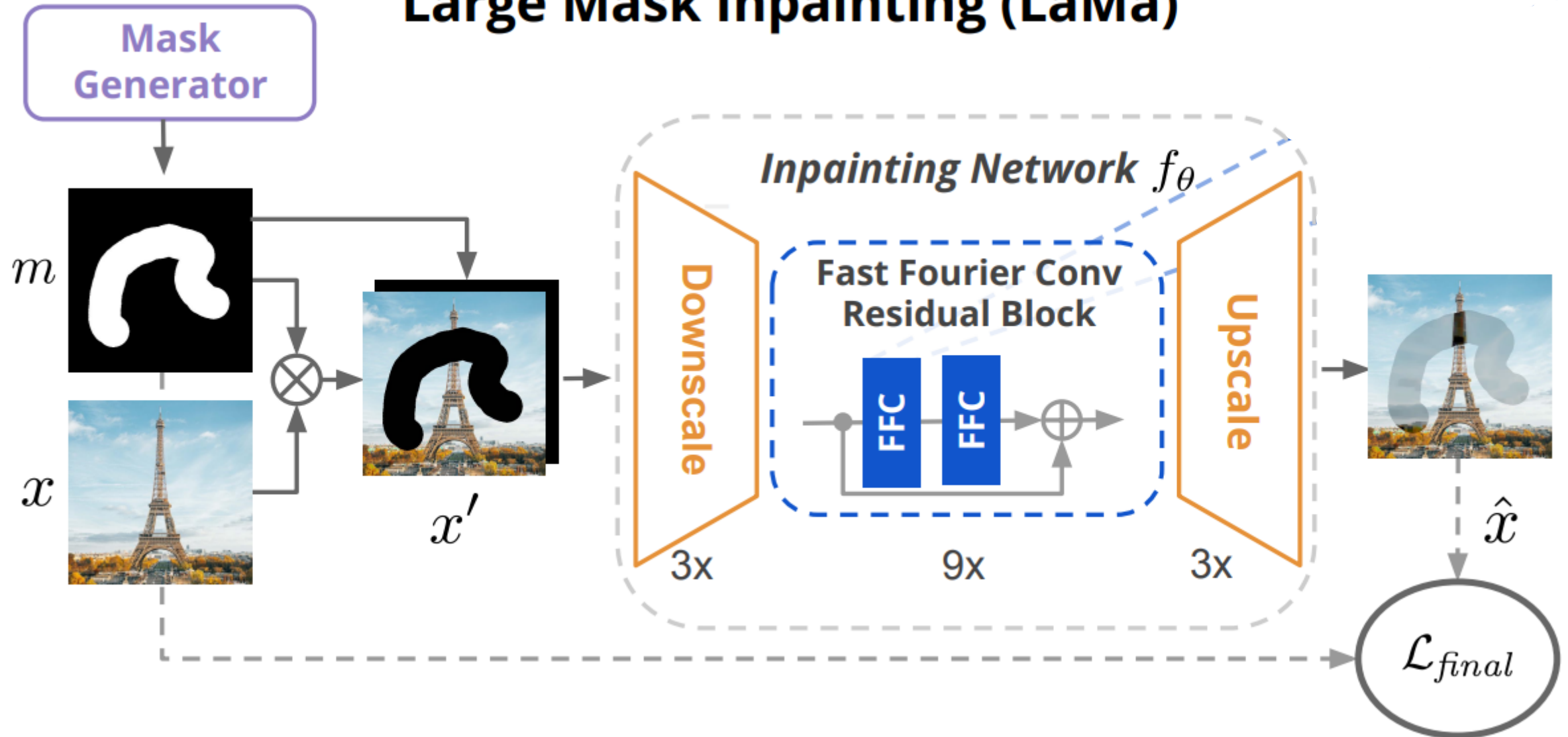


GAN: Inpainting

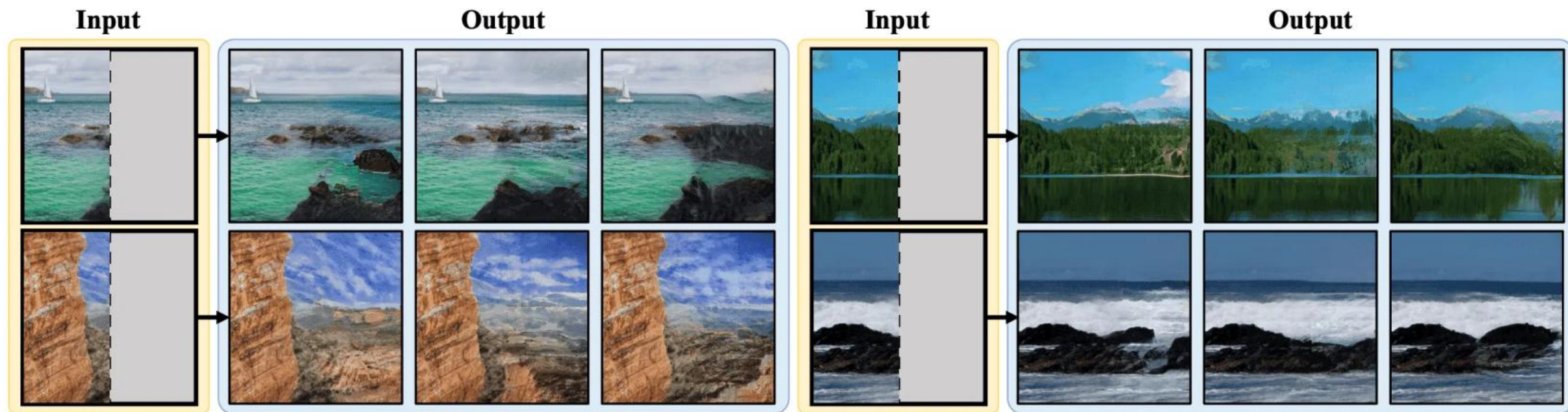


GAN: Inpainting

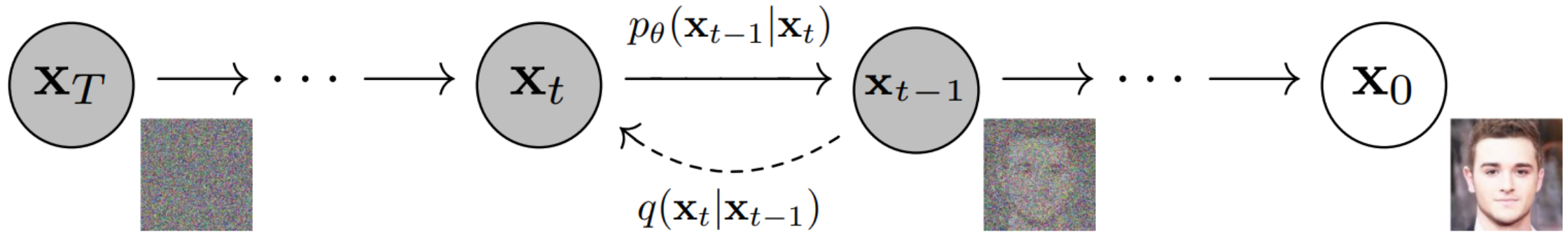
Large Mask Inpainting (LaMa)



GAN: Outpainting



Diffusion Models: What is It?



- a fixed (or predefined) forward diffusion process q of our choosing, that gradually adds Gaussian noise to an image, until you end up with pure noise
- a learned reverse denoising diffusion process p_θ , where a neural network is trained to gradually denoise an image starting from pure noise, until you end up with an actual image

Diffusion Models

Forward diffusion process: $q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$

- With reparameterization trick: $x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon, \epsilon \sim \mathcal{N}(0, I)$
- $q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I), \alpha_t := 1 - \beta_t, \bar{\alpha}_t := \prod_{s=1}^t \alpha_s$

Variance schedule: $0 < \beta_1 < \beta_2 < \dots < \beta_T < 1$

Denoising diffusion process: $p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$

Mean reparameterization: $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$

Diffusion Models

Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_{\theta} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2$   
6: until converged
```

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

Diffusion Models

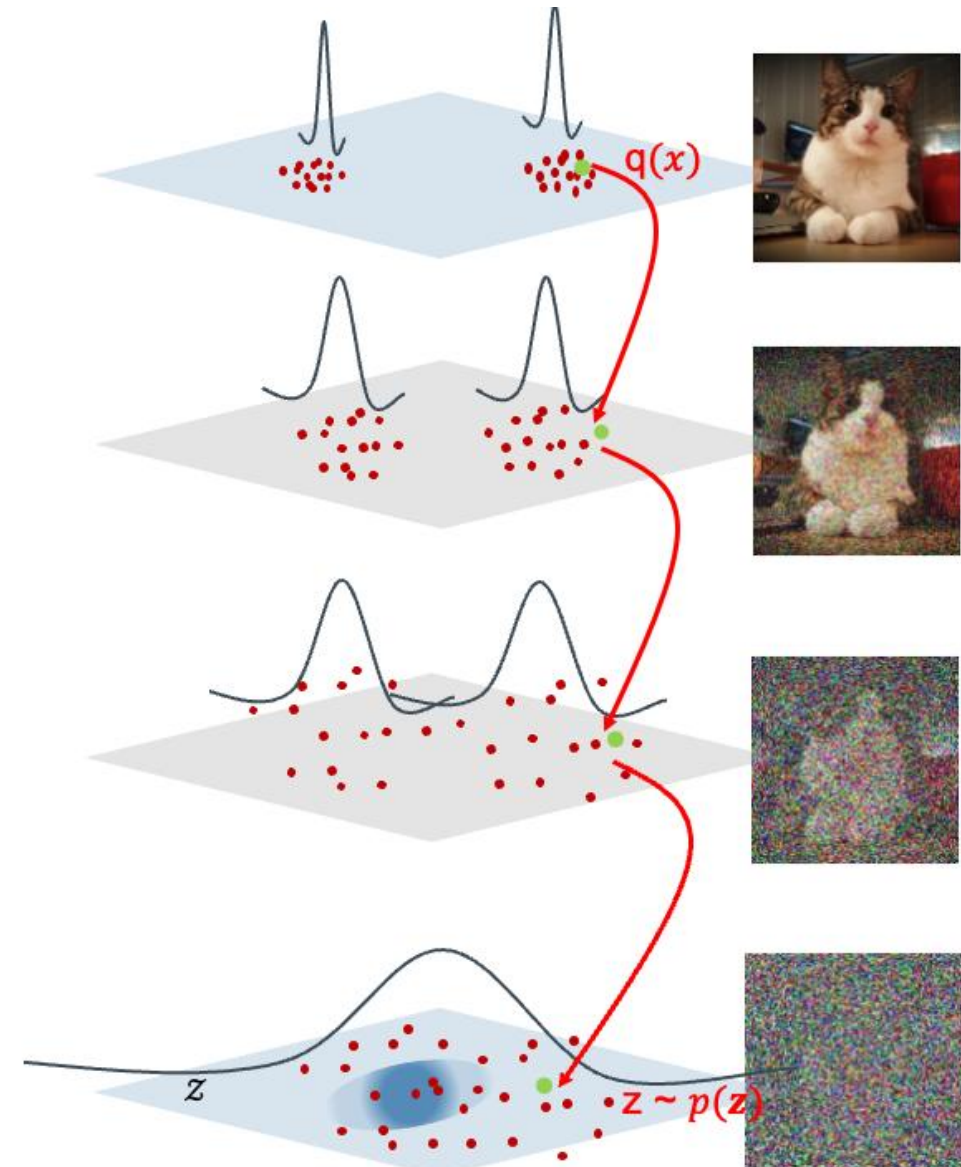
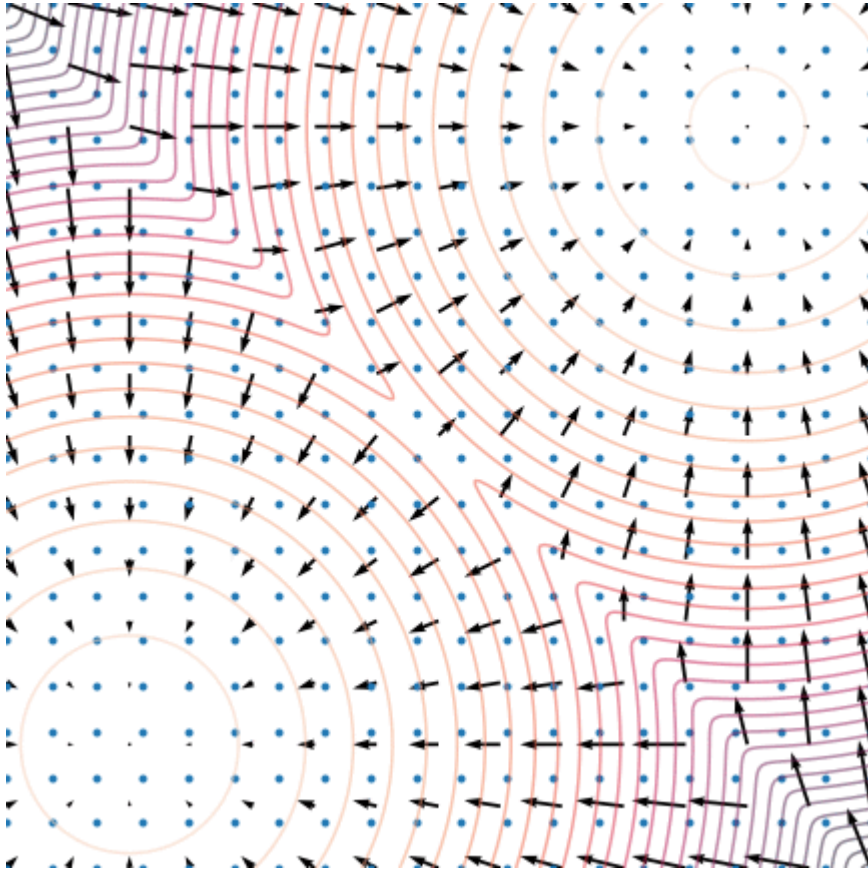
Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_{\theta} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2$   
6: until converged
```

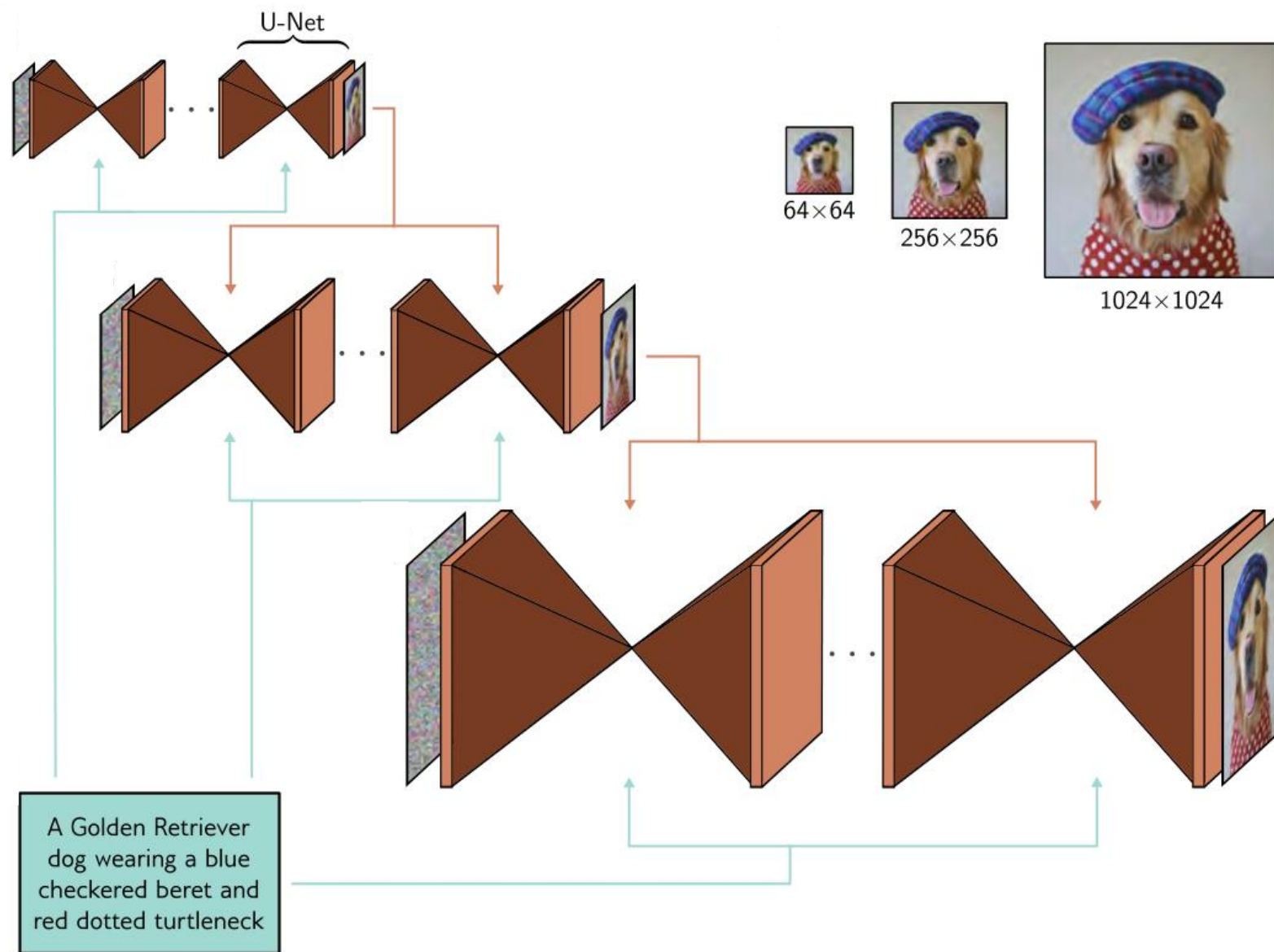
Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

Diffusion Models



Diffusion Models: Examples



Diffusion Models: Examples



Diffusion Models: Examples



A transparent sculpture of a duck made out of glass



An angry duck doing heavy weightlifting at the gym



A brain riding a rocketship heading towards the moon



A couple of glasses sitting on a table



New York skyline with Hello World written with fireworks in the sky

